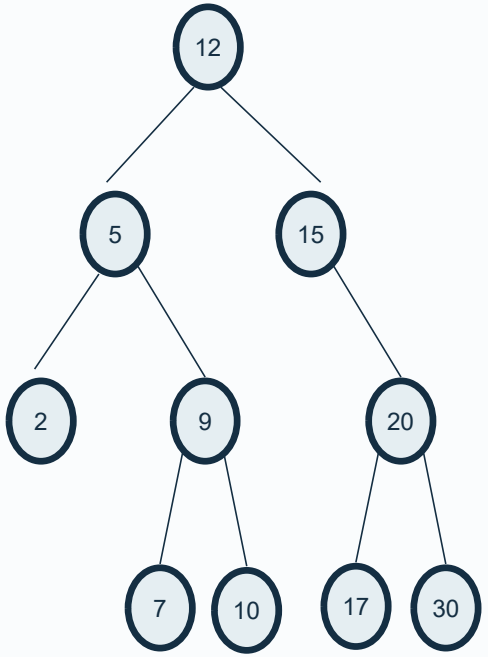
Binary Search Trees

Structure, algorithms, and the cost of height

A lesson in ordered data structures



Learning objectives

Explain the BST ordering rule and tree terminology.

Trace search, insertion, deletion, and traversal.

Read and implement a generic Java BST.

Relate operation cost to the height of the tree.

Why an ordered tree?

Structure	Search by key	Insert / delete
Sorted array	O(log n)	O(n) shifting
Linked list	O(n)	O(n) to locate
BST	O(h + 1)	O(h + 1)

A short tree can support fast search and updates.

Trees and rooted trees

An undirected tree is a connected graph with no cycles.
A forest is an acyclic graph that may be disconnected.

A tree with n nodes has n − 1 edges and exactly one simple path between any two nodes.

Choosing a root defines parent–child relationships.

Tree terminology

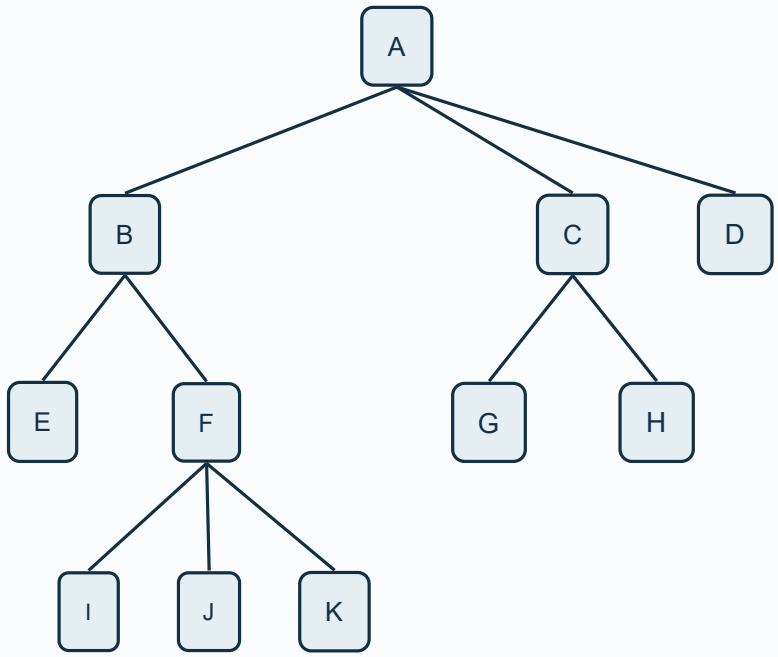
Root: A has no parent.

Leaf: E has no children.

Internal node: B has children.

Ancestors of J: F, B, A.

Subtree at F: F, I, J, K.



Binary trees and binary search trees

A binary tree gives each node at most two children, identified as left and right. Either child may be absent.

A binary search tree adds an ordering rule: every key in the left subtree is smaller, and every key in the right subtree is larger.

Both subtrees obey the same rule.

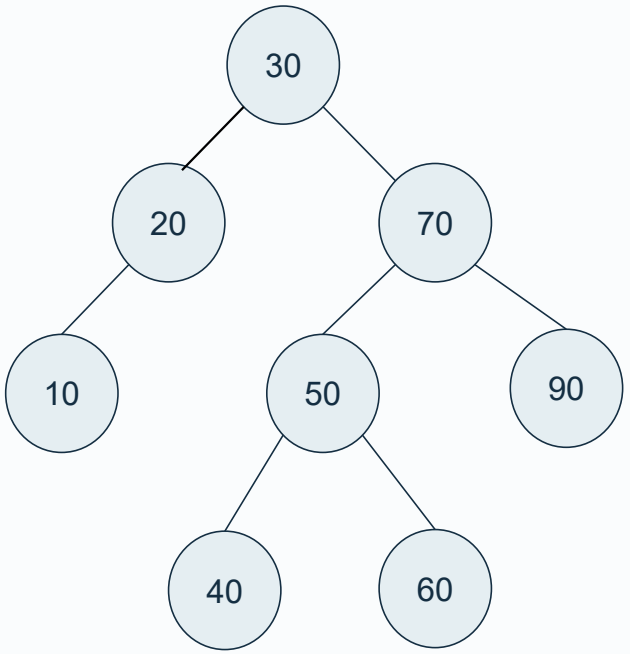
This lesson uses unique keys and rejects duplicates.

Depth, level, and height

Depth counts edges from the root.
Root 30 has depth 0.
Node 40 has depth 3.

Height is the maximum depth.
This tree has height 3 and 4 levels.

Empty tree: height −1.
Single node: height 0.



Full, perfect, complete, and balanced

Term	Definition used in this lesson
Full (proper)	Every node has 0 or 2 children
Perfect	Full, with all leaves at the same depth
Complete	Every level fills, except possibly the last, which fills from left to right
Height-balanced	At every node, subtree heights differ by at most 1

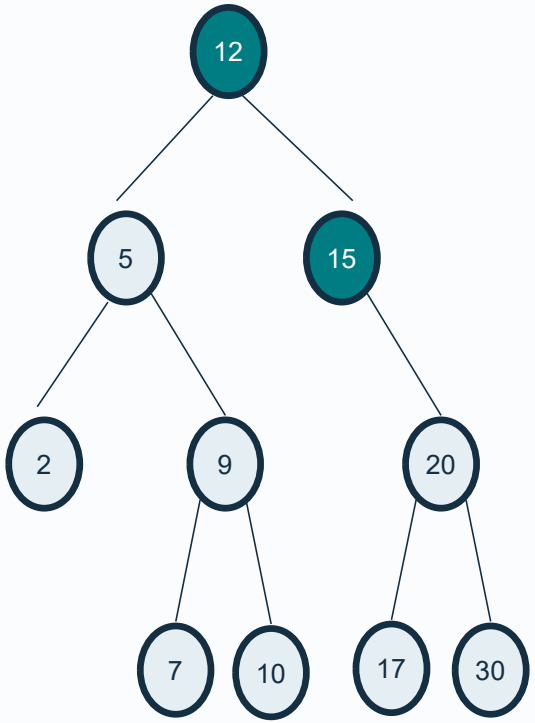
Every perfect tree is complete. Every complete tree is height-balanced.

The BST ordering rule applies to every descendant

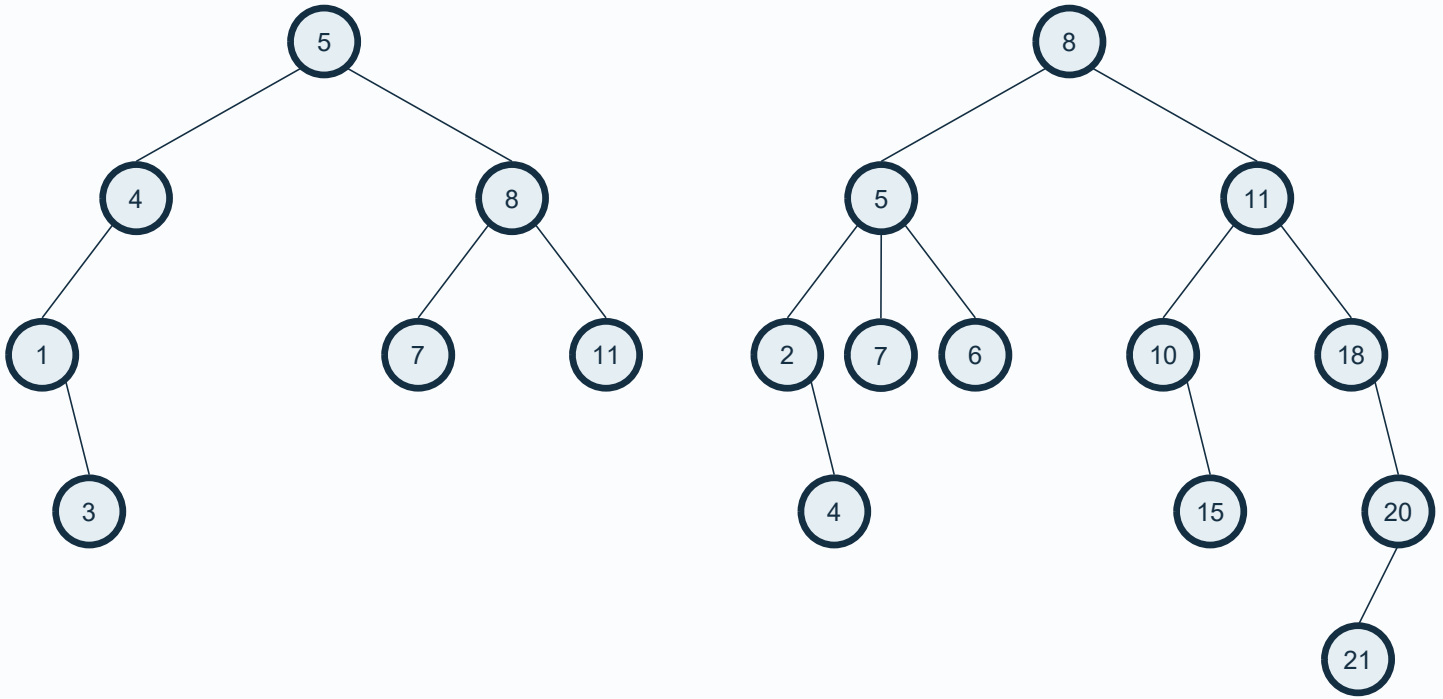
At root 12:
all left keys are less than 12,
all right keys are greater than 12.

At node 15:
all right keys are greater than 15.

A parent–child check alone
cannot validate a whole BST.



Checkpoint: which drawing is a valid BST?



For each node, carry the lower and upper bounds from its ancestors.

A linked representation in Java

```
class Node<E> {
    E key;
    Node<E> left, right;

    Node(E key) { this.key = key; }
}

class BST<E extends Comparable<? super E>> {
    private Node<E> root;
    private int size;
    // Methods follow.
}
```

A missing child uses null. The tree stores a root reference and a size.

The Tree interface and BST implementation

```
interface Tree<E> extends Iterable<E> {
    boolean contains(E key);
    boolean insert(E key);
    boolean delete(E key);
    int size();
}
```

BST<E> implements Tree<E>.
insert returns false for a duplicate key.
delete returns false when the key is absent.

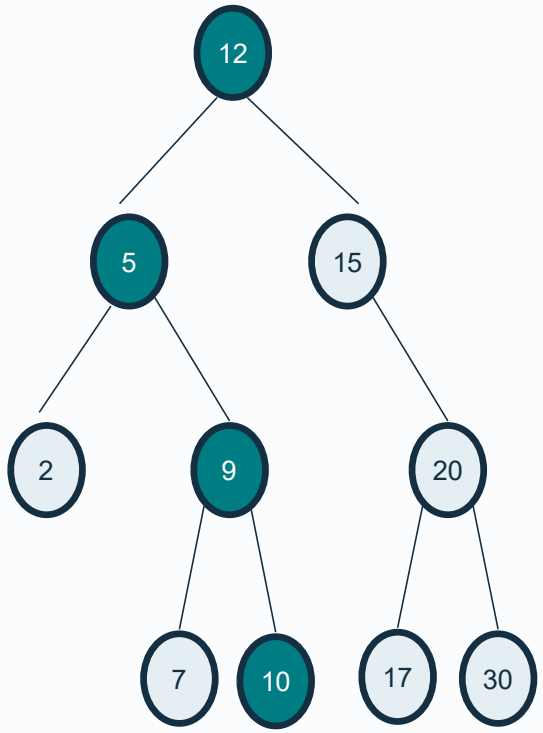
Use compareTo consistently for less than, greater than, and equality.

Search follows one path

Search for 10

10 < 12: go left.
10 > 5: go right.
10 > 9: go right.
10 = 10: found.

A null child ends an unsuccessful search.



Iterative search

```
public boolean contains(E key) {
    Objects.requireNonNull(key);
    Node<E> x = root;
    while (x != null) {
        int cmp = key.compareTo(x.key);
        if (cmp < 0) x = x.left;
        else if (cmp > 0) x = x.right;
        else return true;
    }
    return false;
}
```

Time O(h + 1). Extra space O(1). Import java.util.Objects.

Recursive search

```
private boolean contains(Node<E> x, E key) {
    if (x == null) return false;
    int cmp = key.compareTo(x.key);
    if (cmp < 0) return contains(x.left, key);
    if (cmp > 0) return contains(x.right, key);
    return true;
}
```

The public method checks the key, then calls contains(root, key).

Time O(h + 1). Call stack O(h + 1).

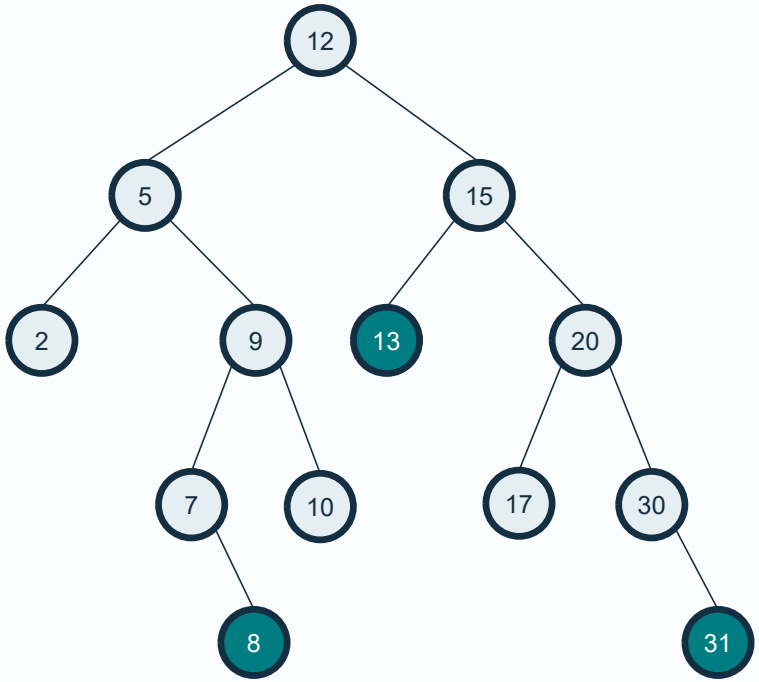
Insertion attaches a new leaf

Search for the new key.

If an equal key exists, return false.

Otherwise, attach a new node
at the first missing child link.

An empty tree gets a new root.



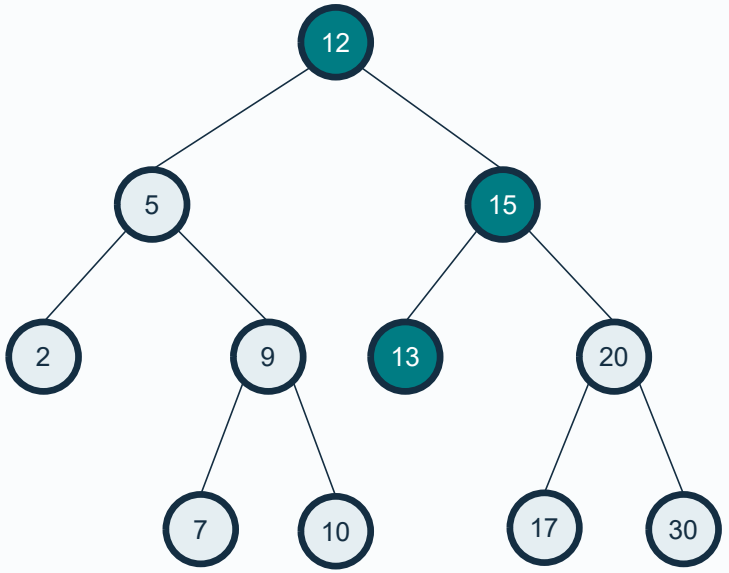
Worked insertion: key 13

Compare with 12: 13 is larger.

Compare with 15: 13 is smaller.

The left link of 15 is empty.
Attach node 13 there.

The BST ordering rule still holds.



Iterative insertion

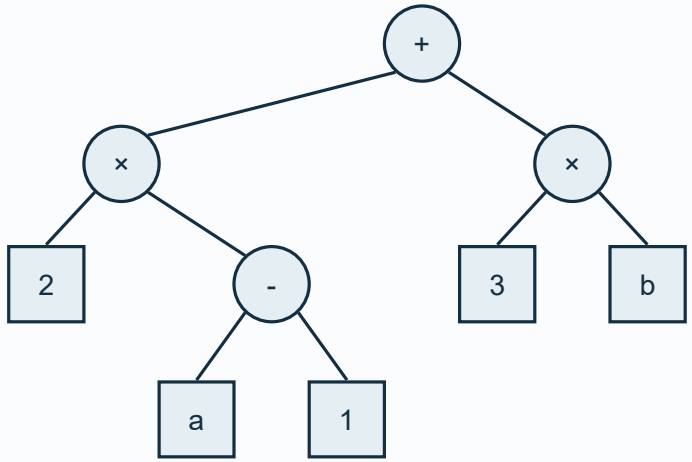
```
public boolean insert(E key) {
    Objects.requireNonNull(key);
    Node<E> parent = null, x = root;
    int cmp = 0;
    while (x != null) {
        parent = x;
        cmp = key.compareTo(x.key);
        if (cmp == 0) return false;
        x = cmp < 0 ? x.left : x.right;
    }
    Node<E> node = new Node<>(key);
    if (parent == null) root = node;
    else if (cmp < 0) parent.left = node;
    else parent.right = node;
    size++;
    return true;
}
```

An expression tree is a binary tree

Expression:
2 × (a − 1) + 3 × b

Internal nodes hold operators.
Leaves hold operands.

The shape captures evaluation order rather than key ordering.

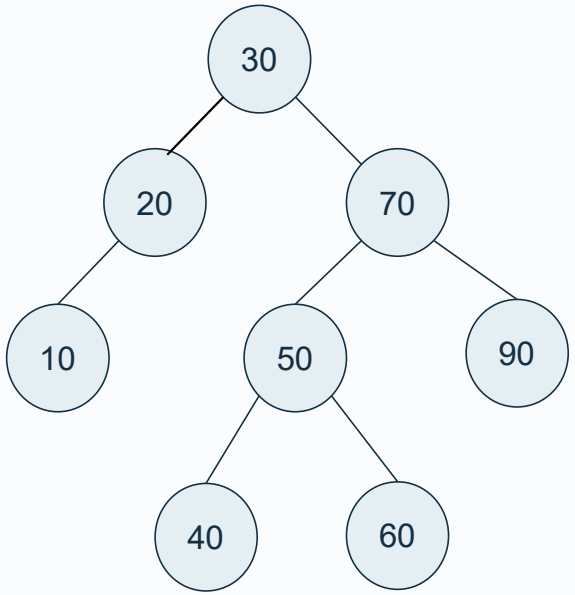


Traversal visits every node once

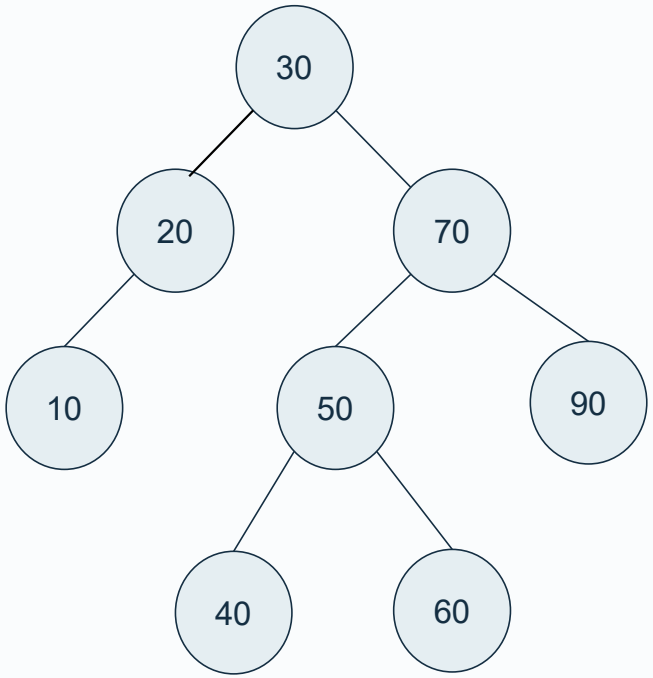
Depth-first search follows a branch before returning to other branches.

Preorder: node, left, right.
Inorder: left, node, right.
Postorder: left, right, node.

Breadth-first search visits nodes one level at a time.



Traversal orders on the same tree



Preorder
30, 20, 10, 70, 50, 40, 60, 90

Inorder
10, 20, 30, 40, 50, 60, 70, 90

Postorder
10, 20, 40, 60, 50, 90, 70, 30

Inorder traversal of a BST lists its keys in sorted order.

Recursive depth-first traversal

```
void inorder(Node<E> x) {
    if (x == null) return;
    inorder(x.left);
    visit(x.key);
    inorder(x.right);
}
```

Preorder moves visit before both recursive calls.
Postorder moves visit after both recursive calls.

visit is the action to perform on a key, such as printing it.

All three take O(n) time and O(h + 1) stack space.

Breadth-first traversal uses a queue

```
void levelOrder() {
    if (root == null) return;
    Queue<Node<E>> q = new ArrayDeque<>();
    q.add(root);
    while (!q.isEmpty()) {
        Node<E> x = q.remove();
        visit(x.key);
        if (x.left != null) q.add(x.left);
        if (x.right != null) q.add(x.right);
    }
}
```

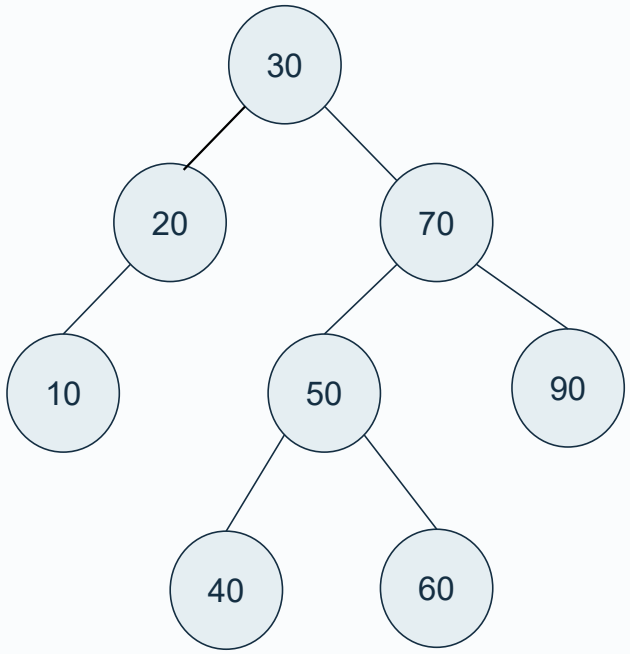
Time O(n). Queue space O(w), where w is the maximum level width.

Checkpoint: traversal and successor

Write the inorder, preorder, and postorder sequences.

Which key follows 30 in inorder traversal?

Explain why that key is the inorder successor.



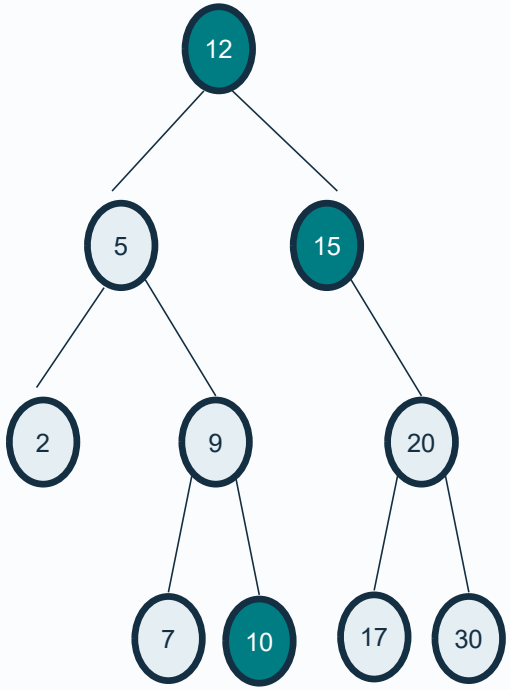
Predecessor and successor

For root 12:

Predecessor = 10
Largest key in the left subtree.

Successor = 15
Smallest key in the right subtree.

The maximum follows right links.
The minimum follows left links.



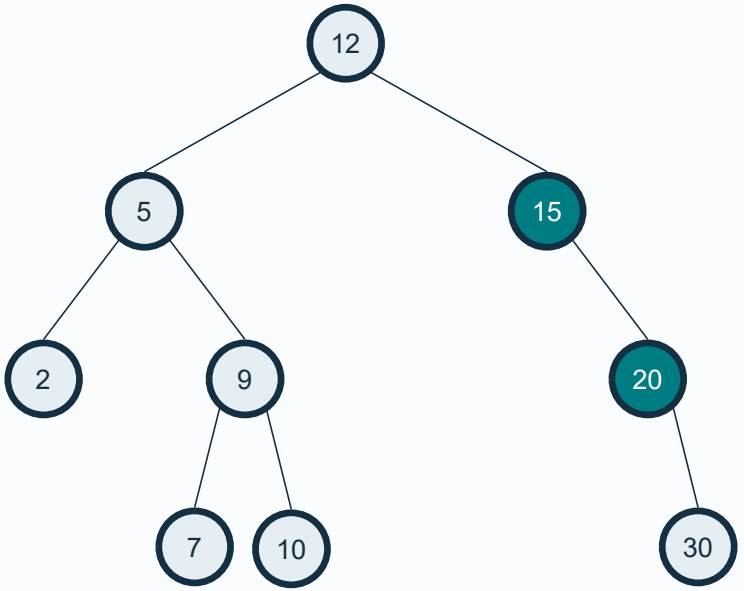
Deletion has three structural cases

Children	Repair after finding the node
0	Remove the parent’s child link
1	Connect the parent directly to the child
2	Copy the successor key, then remove the successor from the right subtree

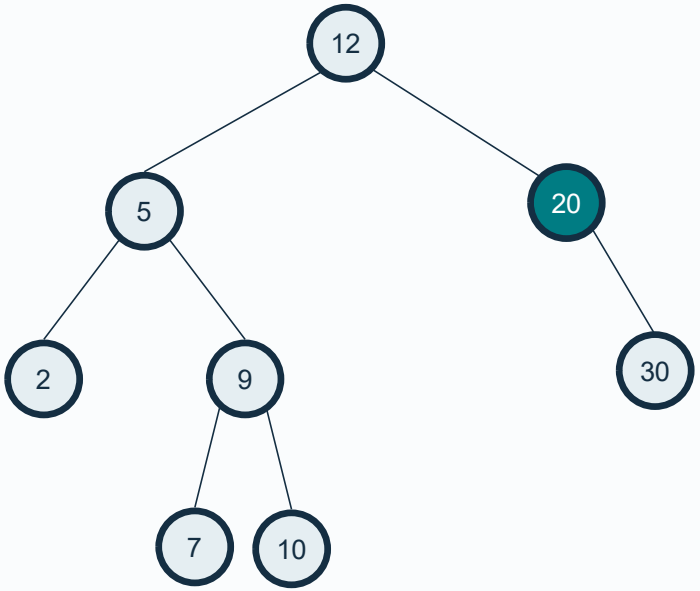
Deleting the root can change root. An absent key leaves size unchanged.

One child: deletion reconnects the subtree

Before: delete 15



After: 12 links to 20



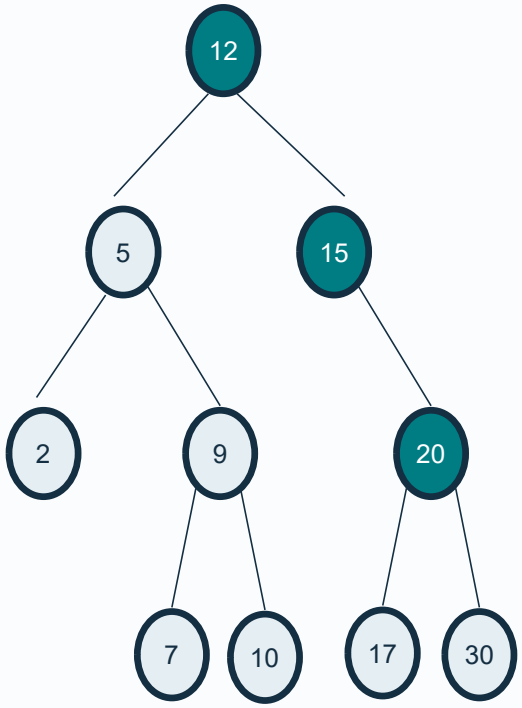
A leaf uses the same reconnection idea with a null replacement.

Two children: deletion uses a neighboring key

Delete 12 using its successor.

1. Find the minimum in the right subtree: key 15.
2. Copy 15 into the root.
3. Remove the original node 15, reconnecting its child 20.

The predecessor is an equally valid alternative.



Recursive deletion preserves the BST rule

```
private Node<E> delete(Node<E> x, E key) {
    if (x == null) return null;
    int cmp = key.compareTo(x.key);
    if (cmp < 0) x.left = delete(x.left, key);
    else if (cmp > 0) x.right = delete(x.right, key);
    else {
        if (x.left == null) return x.right;
        if (x.right == null) return x.left;
        x.key = min(x.right).key;
        x.right = deleteMin(x.right);
    }
    return x;
}
```

Each call reconnects the returned subtree root to its parent.

Successor helpers

```
private Node<E> min(Node<E> x) {
    while (x.left != null) x = x.left;
    return x;
}

private Node<E> deleteMin(Node<E> x) {
    if (x.left == null) return x.right;
    x.left = deleteMin(x.left);
    return x;
}
```

The successor has no left child. It may have a right child to reconnect.

The public deletion method

```
public boolean delete(E key) {
    Objects.requireNonNull(key);
    if (!contains(key)) return false;
    root = delete(root, key);
    size--;
    return true;
}
```

The preliminary search keeps size bookkeeping simple.
A later optimization can combine search and deletion.
Both versions take O(h + 1) time.

An inorder iterator

An iterator exposes hasNext() and next().

A lazy inorder iterator keeps a stack of ancestors.
Initially, push the path from root to the leftmost node.

On next(), pop a node and return its key.
Push the leftmost path in that node's right subtree.

Across n calls, total time is O(n) with O(h + 1) stack space.

Exercise: handle an exhausted iterator with NoSuchElementException.

Height determines operation cost

Operation	General BST	If height is O(log n)
Search, insert, delete	O(h + 1)	O(log n)
Minimum, maximum	O(h + 1)	O(log n)
Full traversal	O(n)	O(n)

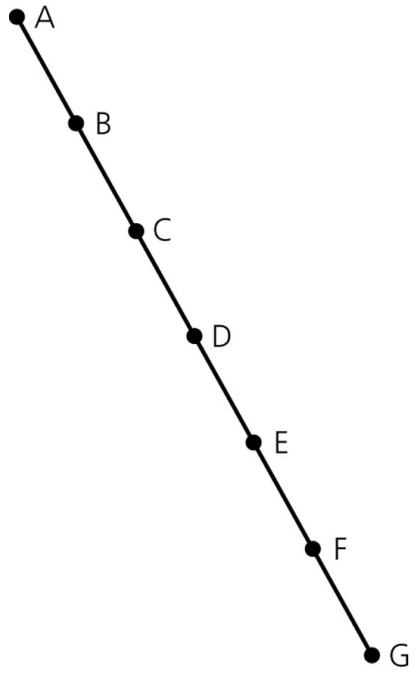
Sorted insertion can create a chain with height n − 1.
A balanced search tree controls height as the data changes.

binary tree time complexity

It is obvious that the time complexity for the inorder, preorder, and postorder is O(n), since each node is traversed only once. The time complexity for search, insertion and deletion is the height of the tree. In the worst case, the height of the tree is O(n).

The Efficiency of Binary Search Tree Operations

- The maximum number of comparisons for a retrieval, insertion, or deletion is the height of the tree
- The maximum and minimum heights of a binary search tree
  - n is the maximum height of a binary tree with n nodes



A maximum-height binary tree with seven nodes

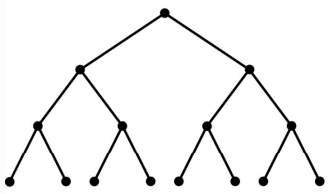
Height bounds with an edge count

For height h ≥ 0, the maximum node count is

2^(h + 1) − 1

For n ≥ 1 nodes, height satisfies

ceil($\log_2$(n + 1)) − 1 ≤ h ≤ n − 1



Level	Number of nodes at this level	Number of nodes at this and previous levels
1	$1 = 2^0$	$1 = 2^1 - 1$
2	$2 = 2^1$	$3 = 2^2 - 1$
3	$4 = 2^2$	$7 = 2^3 - 1$
4	$8 = 2^3$	$15 = 2^4 - 1$
.	.	.
.	.	.
.	.	.
h	2^{h-1}	$2^h - 1$

A perfect tree attains the maximum node count for its height.

Treesort

Insert the input keys into a BST.
Then emit keys in inorder.

Expected time with random insertion order: O(n log n).
Worst case for an unbalanced tree: $O(n^2)$.

A set-style BST discards duplicates. A sorting implementation must retain equal keys, for example with a count per node.

Review problems

1. What is the maximum node count with 8 levels?

2. For 2011 nodes, what are the minimum and maximum numbers of levels?

3. With exactly 10 levels, what are the minimum and maximum numbers of leaves?

4. Why can sorted insertion make a BST slow?

Review answers

1. $2^8 - 1 = 255$ nodes.

2. Minimum: ceil($\log_2$ 2012) = 11 levels.
   Maximum: 2011 levels, a chain.

3. Minimum: 1 leaf. Maximum: $2^9 = 512$ leaves.

4. Each key can extend the same branch, creating height n − 1 and linear search paths.

Further practice

Implement iterative inorder traversal with an explicit stack.

Compute height using a queue and one level at a time.

Validate a BST using lower and upper bounds.

Compare sorted insertion with random insertion.

Explain how changing a stored key could break ordering.

A BST’s ordering rule gives the algorithm. Its shape determines the cost.